

Hardware Modeling using Verilog

Lecture 01 — Introduction

Tutorial and Practice Workbook

Prof. Indranil Sengupta · Department of Computer Science and Engineering · IIT
Kharagpur (NPTEL)
expanded and made hands-on for the Chip Design Associate programme · Module 2 ·
NOS NIE/ELE/N0102

How this workbook is organised

- Part 1** The lecture itself, written out in full. Every slide the lecturer shows is covered, in his order, with the explanation expanded so that nothing has to be looked up elsewhere.
- Part 2** The design flow in detail: what each step takes in, what it produces, and which tool does it.
- Part 3** Installing the toolchain, on Linux, macOS or Windows, with the checks that tell you it worked.
- Part 4** Five guided labs. Every command is given, and every output printed here was produced by actually running it.
- Part 5** Thirty-six exercises with full worked solutions, and a reference section.

Before you start

You need a computer you can install software on, and about an hour. No licences, no university server and no vendor account are required: every tool used in this workbook is open source.

Everything in Parts 4 and 5 lives in `ChipDesignAssociate/VerilogCourse/Lecture01_Lab/` in the course repository.

Part 1 · The Lecture

This part follows Lecture 01 slide by slide, in the lecturer's own order. Where he says something briefly, it is expanded; where he refers to something you may not know, it is explained. Nothing has been reordered.

1.1 What this course is for

The lecturer opens with six objectives. They are worth reading slowly, because between them they describe everything the next eight weeks will do.

Hardware Modeling Using Verilog — the lecturer's own six objectives, with what each one will mean in practice

1	Learn about the Verilog hardware description language. Syntax, data types, operators, and the module as the unit of design.
2	Understand the difference between behavioral and structural design styles. Say WHAT the circuit does, or name WHICH PARTS it is built from. This distinction runs through the course.
3	Learn to write test benches and analyze simulation results. A design you cannot test is a design you cannot trust. Weeks 5 onwards are built on this.
4	Learn to model combinational and sequential circuits. Logic with no memory, and logic with memory. Almost every real block is a mixture of the two.
5	Distinguish between good and bad coding practices. Verilog will let you write something that simulates correctly and synthesises into the wrong hardware.
6	Case studies with some complex designs. Week 8 builds a pipelined processor — datapath and control path — entirely in Verilog.

The six objectives, with what each will mean in practice

Objective 1 — learn the Verilog hardware description language

Verilog is a language in which a designer specifies the behaviour, the functionality or the structure of a hardware circuit. You will learn its syntax, its data types, its operators, and the module as the unit of design.

Objective 2 — behavioural and structural design styles

These are two different ways of saying what a circuit is, and the distinction runs through the entire course. A behavioural description says WHAT the circuit does. A structural description names WHICH PARTS it is built from and how they are wired together. Both are legal Verilog; both can describe the same circuit; and a synthesis tool turns the first into the second.

Objective 3 — test benches and simulation results

A design you cannot test is a design you cannot trust. A test bench is a piece of Verilog whose job is to drive your design with stimulus and check what comes out. It is never turned into hardware, so it may use delays, loops and printing freely. Week 5 of the course is devoted to this; you will write your first one in Lab 1 of this workbook.

Objective 4 — combinational and sequential circuits

Combinational logic has no memory: its outputs depend only on its present inputs. Sequential logic has memory: its outputs depend also on what happened before, which is held in registers and updated on a clock edge. Almost every real block is a mixture of the two.

Objective 5 — good and bad coding practices

Verilog will happily let you write something that simulates correctly on your machine and synthesises into hardware that does something different. Week 6 is about avoiding that, and the reason it needs a whole week is the subject of the next section.

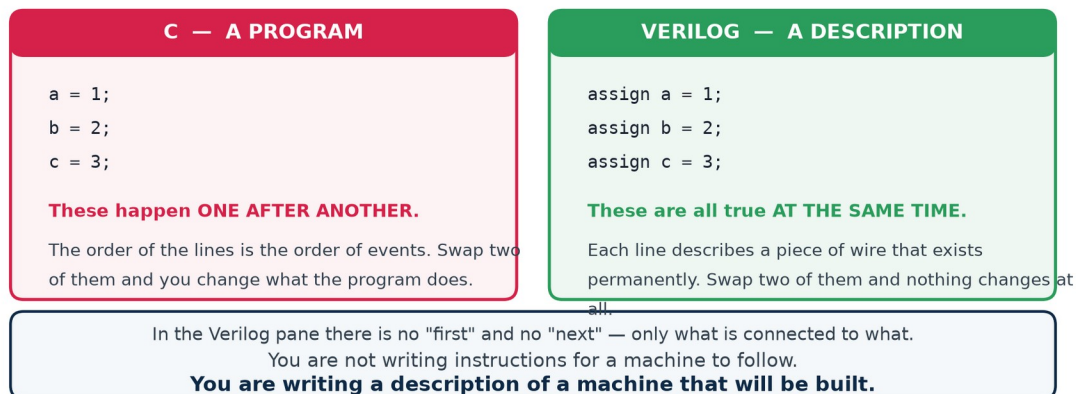
Objective 6 — case studies

Week 8 builds a pipelined processor — both its data path and its control path — entirely in Verilog. Everything before it is preparation for that.

1.2 What a hardware description language is — and is not

This is the single most common beginner error, so it is worth stating plainly before anything else: **Verilog is not a programming language.** It looks like one. It is text, it has semicolons, it has if-statements. But what the text MEANS is completely different.

Both are text you type. What the text MEANS is completely different — and this is the single most common beginner error.



Three lines in C, and three lines in Verilog

In C, three assignments happen one after another. The order of the lines is the order of events; swap two of them and you change what the program does.

C — a sequence of events in time

```
a = 1;
b = 2;
c = 3;
```

In Verilog, those same three lines describe three pieces of wire, each permanently driven. All three statements are true at every instant the circuit is powered. There is no "first" and no "next". Swap two of them and nothing changes at all.

Verilog — a description of three wires that exist

```
assign a = 1;  
assign b = 2;  
assign c = 3;
```

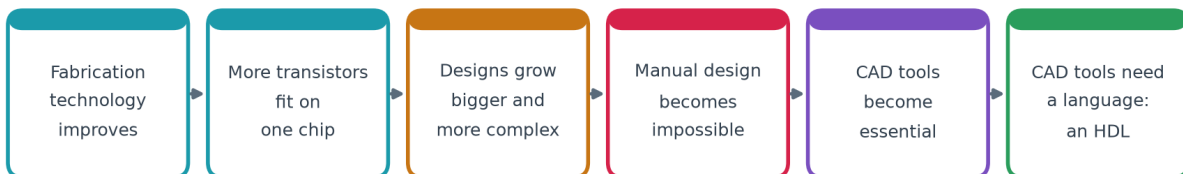
Hold on to this

You are not writing instructions for a machine to follow. You are writing a description of a machine that will be built. Once you really believe that, most of the confusion in Weeks 2 and 3 of the course never happens.

Part 2 · The Problem, and the Answer

2.1 Why VLSI design had to become a process

A hardware description language only exists because of a problem, and the lecturer spends the first third of the lecture on that problem. It is a chain of consequences, each one caused by the one before it.



And the requirements conflict with each other

Smaller area, higher speed, lower energy — you cannot have all three. Push on one and at least one of the others gets worse. That is why the design flow is not a recipe to follow but a series of engineering choices to make.

Each box is caused by the one to its left

Fabrication technology improved, so more transistors fitted on one chip. Designs therefore grew bigger and more complex. Past a certain size, designing them by hand became impossible, so computer-aided design tools became essential — and a CAD tool needs a language to read and write. That last link is where Verilog enters, and it is the only reason this course has a subject.

How big is "too big"?

The lecturer puts two chips side by side: the first commercial planar integrated circuit, and a modern processor.

The two chips the lecturer puts side by side — and the factor between them

FIRST COMMERCIAL PLANAR IC	A MODERN PROCESSOR	TODAY, FOR COMPARISON
Fairchild Micrologic flip-flop, 1961	Intel Core i7 (Nehalem) quad-core, 2008	Apple M1 Ultra, 2022
about 4 transistors	731 000 000 transistors	114 000 000 000 transistors

What that factor actually means for a designer

From 4 devices to 731 million is a factor of about 180 million. If you could place and wire one transistor per second, by hand, without ever sleeping, the Nehalem alone would take you about 23 years.

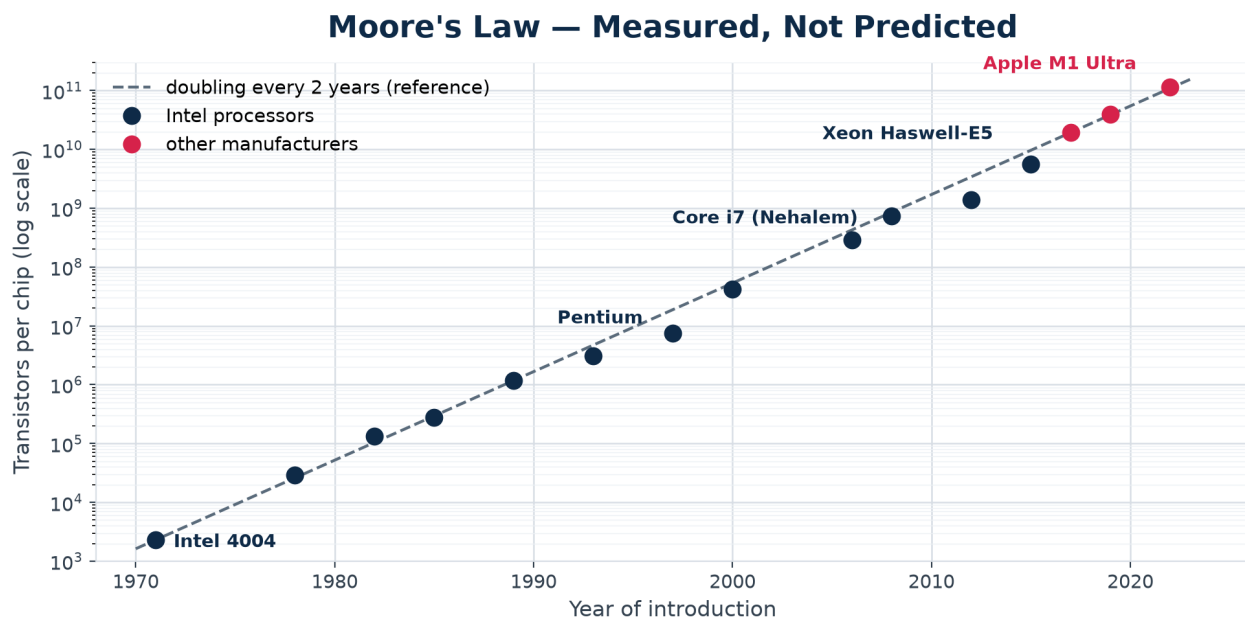
The arithmetic that settles it

If you could place and wire one transistor per second, by hand, without ever sleeping, the 731-million-transistor Nehalem die would take you about twenty-three years. And then one change to the specification would send you back to the beginning.

That is what the lecturer means by **"manual design is simply ruled out"**. The job of a VLSI designer changed: a designer no longer places transistors, but writes a DESCRIPTION of what the circuit must do, and a program turns that description into transistors.

2.2 Moore's Law

The growth that caused all this has a name. In 1965 Gordon Moore noticed that the number of components on a chip was doubling every year; in 1975 he revised it to every two years. That is Moore's Law, and it has held for about five decades.



Published transistor counts, 1971–2022, on a log scale. A straight line on a log scale means constant exponential growth.

It is not a law of physics

This matters more than it sounds. Moore's Law was an observation about what the industry had been doing, and it kept coming true largely because it became a target: equipment makers, materials suppliers and chip companies all planned their roadmaps around it, so the investment arrived on schedule to make the next node happen. A self-fulfilling forecast, not a force of nature.

WHAT MOORE ACTUALLY SAID

In 1965 Gordon Moore looked at four data points and saw components per chip doubling every year. In 1975 he revised it to every two years. It was a note on what the industry had been doing.

WHY IT KEPT COMING TRUE

Because it became a target. Equipment makers, materials suppliers and chip companies all planned around it, so the investment arrived on schedule. A self-fulfilling forecast.

WHY IT IS SLOWING

Transistors are now a few tens of atoms across. Leakage, heat and the cost of each new fab rise faster than the gain. The answer has shifted to chipleths, 3-D stacking and accelerators.

For this course the point is simply this: the number of things on a chip grew far faster than any human's ability to place them. Everything that follows — the flow, the tools, the language — exists to close that gap.

What a nanometre is

The lecture quotes "CMOS up to 22 nm, FinFET 14 nm", which was correct when it was recorded. Those node names are now marketing labels rather than measurements — no feature on a "3 nm" chip is 3 nm across. But the physical scale is worth feeling.

A sense of scale, so the node numbers mean something physical

A human hair	about 70 000 nm wide	
A red blood cell	about 7 000 nm across	
Visible light	400 - 700 nm wavelength	
A 22 nm transistor gate	the lecture's "state of the art"	
A silicon atom	about 0.2 nm across	

(bar lengths are illustrative — the real range spans five orders of magnitude)

So a 22 nm gate is roughly 100 silicon atoms across

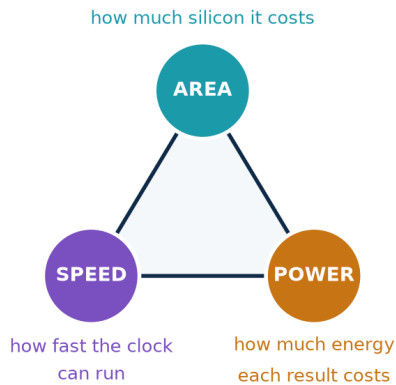
At that size a handful of stray atoms changes a transistor's behaviour. That is why fabrication is statistical, why every chip is tested individually, and why the flow verifies at every level.

A 22 nm gate is roughly a hundred silicon atoms across. At that size a handful of stray atoms changes a transistor's behaviour, which is why fabrication is statistical rather than exact, why every chip is tested individually, and why the design flow carries a verification step at every single level.

2.3 The requirements that conflict

If one design were simply best, a tool could find it and the job would be done. The reason design is hard — and the reason it needs a human — is that the three things you want pull against each other.

"Often these requirements are conflicting. If you try to optimize one you may make the other one worse."



Want less AREA?

Share one adder between several operations — so the job now takes more clock cycles. Speed falls.

Want more SPEED?

Add pipeline registers, or duplicate logic so it runs in parallel. Area rises, and so does switching power.

Want less POWER?

Lower the supply voltage, or gate the clock. Gates get slower, and the control logic gets bigger.

If you want...	You might...	And this gets worse
Less AREA	share one adder between several operations	speed — it now takes more clock cycles
More SPEED	add pipeline registers, or duplicate logic	area, and switching power
Less POWER	lower the supply voltage, or gate the clock	speed, and control-logic area

Why this matters for synthesis

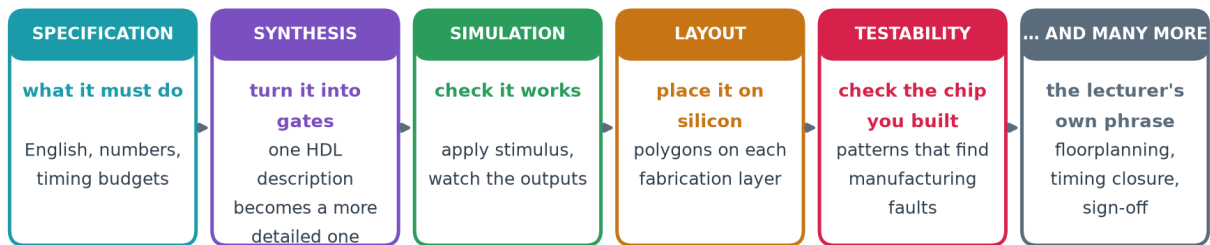
A synthesis tool does not find THE answer, because there isn't one. It finds an answer that satisfies the constraints you gave it. Give it different constraints and you get different hardware out of the same Verilog — which you will see for yourself in Lab 4.

Part 3 · The Design Flow, Step by Step

3.1 What a design flow is

A VLSI design flow is a standardized set of design procedures: a step-by-step route from a specification down to the hardware you actually build. Each step has a defined input, a defined output and a defined way of being checked.

"Step by step, starting from a given specification, down to the actual hardware circuit."



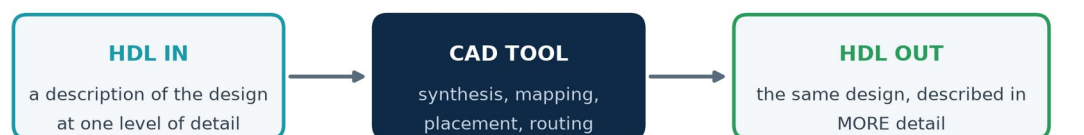
Why it had to be standardized

Dozens of engineers, several companies and a fabrication plant all work on one chip. A standard flow means the work can be divided up, and that a mistake is caught at the step that made it rather than three steps later.

3.2 What a CAD tool actually does

This is the central idea of the whole lecture, and it is worth memorising in the lecturer's own words: **a CAD tool transforms its HDL input into an HDL output that contains more detailed information about the hardware.**

"A CAD tool transforms its HDL input into an HDL output that contains more detailed information about the hardware."



And the tool is applied again and again, one level at a time



Read that carefully. Both the input and the output are hardware descriptions. Nothing is ever "compiled into a program". The design simply gets described in progressively more concrete terms, until the last description is a set of shapes a factory can print.

This transformation	Turns this	Into this
Synthesis	behaviour	register transfer level
Technology mapping	register transfer level	gates
Cell design	gates	transistors
Layout	transistors	polygons on fabrication layers

You will check this yourself

Lab 2 runs exactly these transformations on a ten-line counter, prints all three descriptions side by side, and then runs the same test bench against the first and the last to show they behave identically.

3.3 The two competing HDLs

Two hardware description languages are in common use: Verilog and VHDL. Both can describe the same hardware, and a synthesis tool will produce the same gates from either. The choice between them is a matter of house style, existing code and local convention — not of what the silicon can do.

VERILOG	VHDL	SystemVerilog	SystemC
C-like syntax, terse, and the language of this course. Dominant in industry in the Americas and much of Asia. Standardised as IEEE 1364.	Ada-like syntax, verbose and strongly typed. Strong in Europe, aerospace and defence. Standardised as IEEE 1076.	A large superset of Verilog, adding features aimed mostly at verification: classes, constrained random stimulus, assertions. IEEE 1800.	A C++ class library rather than a separate language, used for modelling whole systems before the hardware is written.

There are others. SystemVerilog is a large superset of Verilog aimed mostly at verification; SystemC is a C++ class library used for modelling whole systems before the hardware is written. This course uses Verilog.

The same counter in both languages

[rtl/counter4.v](#)

```
// Verilog
module counter4 (
    input wire    clk,
    input wire    rst,
    output reg [3:0] q
);
    always @(posedge clk) begin
```

```

        if (rst) q <= 4'd0;
        else    q <= q + 4'd1;
    end
endmodule

```

vhdl/counter4.vhd — the same hardware

```

-- VHDL
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity counter4 is
    port (clk : in  std_logic;
          rst : in  std_logic;
          q   : out std_logic_vector(3 downto 0));
end entity counter4;

architecture rtl of counter4 is
    signal cnt : unsigned(3 downto 0) := (others => '0');
begin
    process (clk)
    begin
        if rising_edge(clk) then
            if rst = '1' then cnt <= (others => '0');
            else               cnt <= cnt + 1;
            end if;
        end if;
    end process;
    q <= std_logic_vector(cnt);
end architecture rtl;

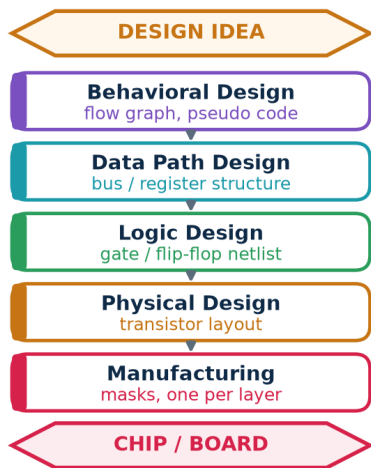
```

VHDL is more verbose and far more strongly typed: the count is held as an unsigned value and has to be converted explicitly before it leaves the entity. Lab 5 simulates both and compares the waveforms.

3.4 The ladder

The lecturer's "simplistic view of the design flow" is a ladder. Each box is a description; each arrow is a CAD tool.

The lecture's own diagram — each box is a description, each arrow is a CAD tool



Each box is a DESCRIPTION

Not a stage of manufacture — a document that says what the circuit is, in a different vocabulary each time.

Each arrow is a TOOL

A program reads the description above it and writes the one below, adding detail that was not there before.

Detail only ever increases

You never climb back up. Each level fixes decisions that the level above deliberately left open.

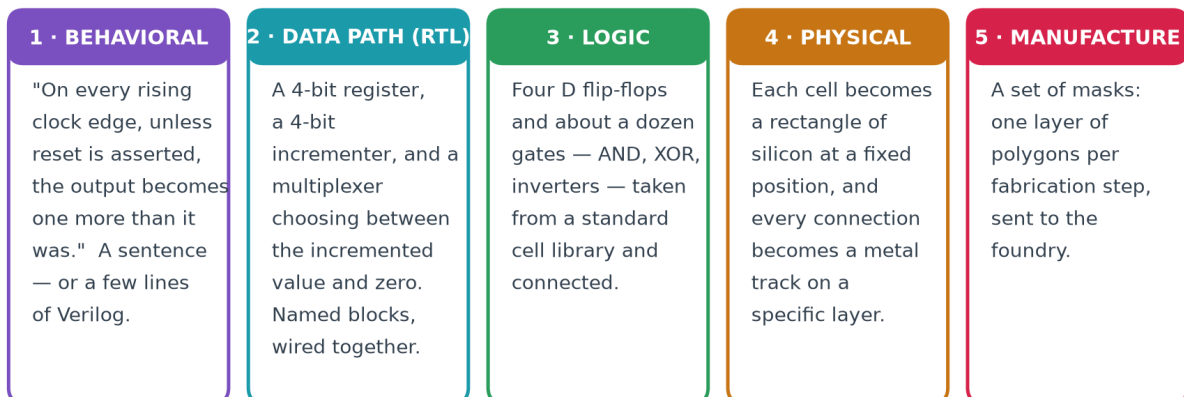
This course lives at the top two

Behavioral and data path — exactly where Verilog is written. The rest is done for you by tools.

Three things are worth noticing. Each box is a document, not a stage of manufacture — it says what the circuit is, in a different vocabulary each time. Each arrow is a program that reads the description above it and writes the one below, adding detail that was not there before. And you never climb back up: each level fixes decisions that the level above deliberately left open.

One circuit, all five levels

A 4-bit counter — the same hardware, described five ways



All five describe exactly the same 4-bit counter. A designer writes the first one. Tools produce the other four.

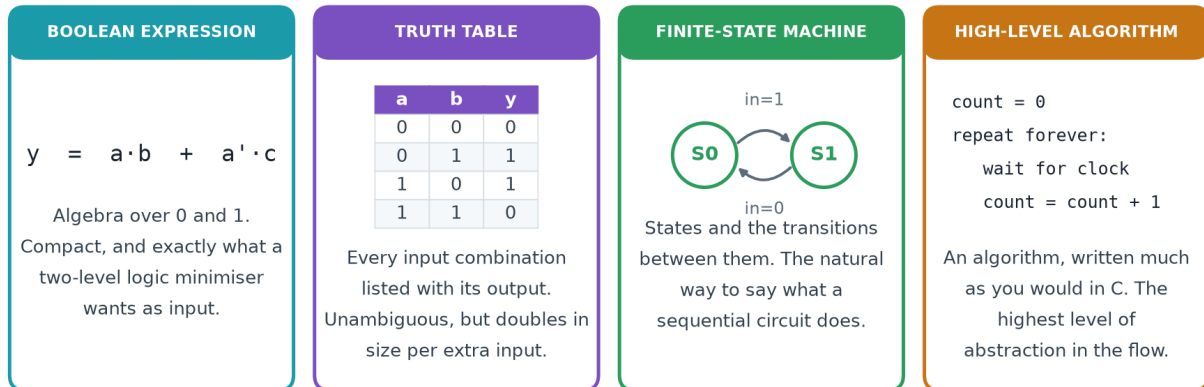
What counts as a "component" changes at each level

Level	A component is...	A connection is...	Typical count for a small design
Behavioral	a statement	a variable name	tens of lines
Data path (RTL)	register, adder, mux	a named bus or wire	tens of blocks
Logic	gate or flip-flop	a net	hundreds of cells
Physical	a placed cell	a metal track	thousands of shapes
Manufacturing	a polygon	a layer	millions of shapes

Read the last column. The number of things to keep track of grows by roughly a factor of ten at every level, which is exactly why a designer works at the top of the table and lets tools handle the bottom.

3.5 Behavioral design

At this level you specify the functionality of the design in terms of its behaviour. You say what you want, not how it is done. There are four common ways of writing it down.



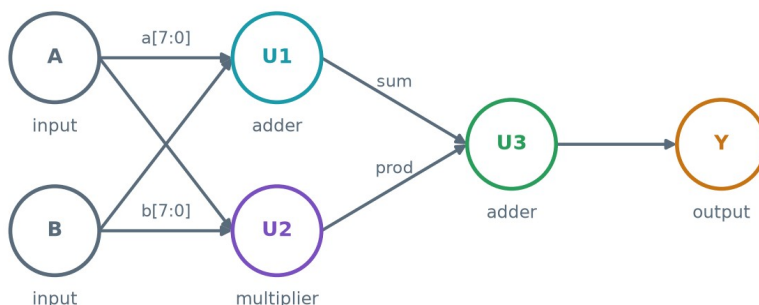
Not one of these says how the circuit is BUILT. That is what makes them behavioural, and it is why a synthesis tool is free to build them in whichever way is cheapest under the constraints you set.

3.6 Data path design, and what a netlist is

One step down, the components are no longer statements. They are registers, adders, multipliers, multiplexers, decoders and buses — things you could point at on a block diagram. This level is called register transfer level, and the name says what happens: on each clock edge, values are TRANSFERRED between REGISTERS, possibly through some logic on the way.

The output of this step is a netlist.

"The vertices indicate components, and the edges indicate interconnections."



VERTEX

one component

EDGE

a wire between two

THE WHOLE GRAPH

the design itself

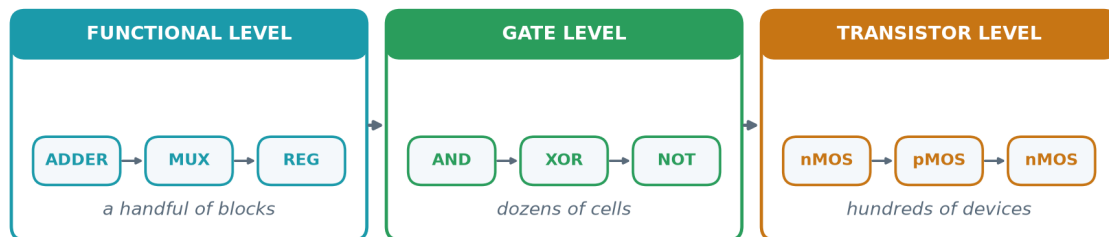
A netlist, defined

A netlist is a **directed graph**. Its vertices are components; its edges are the interconnections between them. The whole graph is the design.

Why a graph rather than a drawing? Because a graph is something a program can hold in memory and transform. Every CAD step after this one — optimisation, technology mapping, placement, routing — is an operation on this graph.

A netlist can be written at several levels. What changes is only what counts as a vertex.

"Netlist may be specified at various levels, where the components may be functional modules, gates or transistors."



synthesis moves you left to right — never the other way

Structural and behavioural, again

The lecturer says it explicitly: a netlist specification is also referred to as a structural design. The two words mean the same thing, and that gives the clearest possible statement of what synthesis is.

BEHAVIOURAL — say WHAT it does

You describe the result. "The output is the sum of the two inputs." You say nothing about which gates are used, or how many. The synthesis tool chooses.

STRUCTURAL — say WHAT IT IS MADE OF

You name the parts and wire them together. "Instantiate a half adder here, another there, and an OR gate across their carries." This is a netlist, written as text.

Synthesis, in one sentence

You write behavioural Verilog; the tool emits structural Verilog.

Here is the same two-bit adder written both ways. Both are legal Verilog and both describe the same hardware.

one line; the tool picks the gates

```
// BEHAVIOURAL - say what it does
module add2 (input a, b, cin, output sum, cout);
```

```
    assign {cout, sum} = a + b + cin;
endmodule
```

five named gates, wired together — a netlist written as text

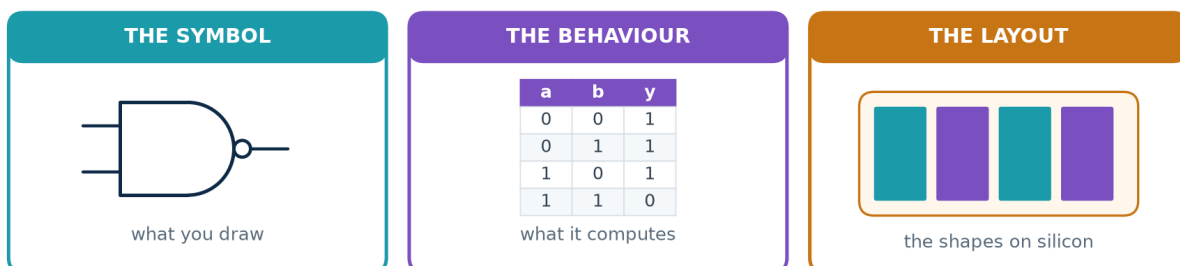
```
// STRUCTURAL - say what it is made of
module add2 (input a, b, cin, output sum, cout);
    wire s1, c1, c2;
    xor u1 (s1, a, b);
    and u2 (c1, a, b);
    xor u3 (sum, s1, cin);
    and u4 (c2, s1, cin);
    or u5 (cout, c1, c2);
endmodule
```

You will write far more behavioural code than structural. But the tool's output is always structural, which is exactly why you have to be able to read it.

3.7 Logic design and standard cells

One step down again, the netlist's components become gates and flip-flops — or, more usefully, standard cells.

"A pre-designed circuit module — like gates, flip-flops, a multiplexer — at the layout level."

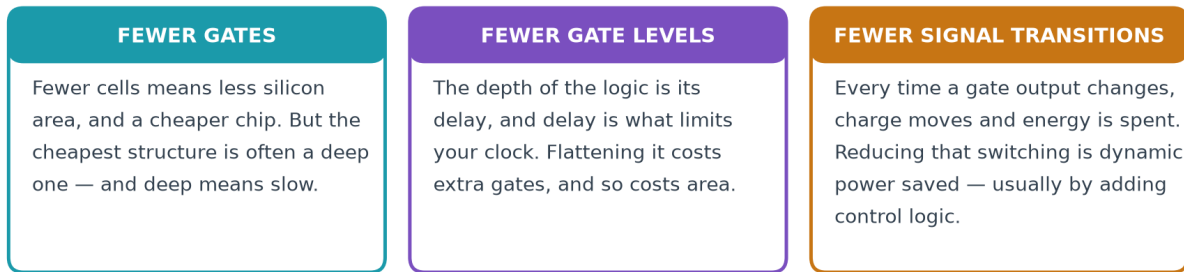


What a standard cell is

A pre-designed circuit module — a gate, a flip-flop, a small multiplexer — held in a library as all of these at once: a symbol to place, a truth table to simulate, a timing model, and a finished piece of layout.

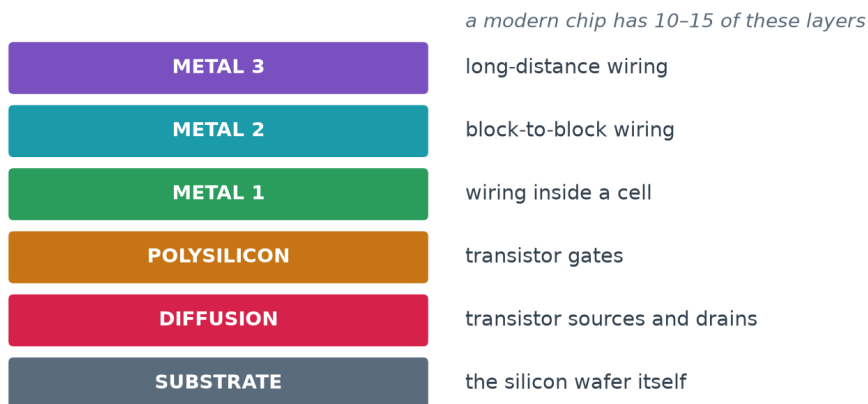
Laying out a NAND gate correctly takes a skilled engineer a long time. Doing it once, perfectly, and then reusing it a million times is the only way the numbers work.

At this level various logic optimisation techniques are applied to obtain a cost-effective design. The lecturer warns that the goals conflict, and names three.



3.8 Physical design and manufacturing

The last step generates the layout that goes to the foundry. At this level the design is a very large number of regular geometric shapes — mostly rectangles — one set per fabrication layer.



They are rectangles because they are printed photographically, layer by layer, onto a silicon wafer: regular shapes expose cleanly and etch predictably, awkward ones do not.

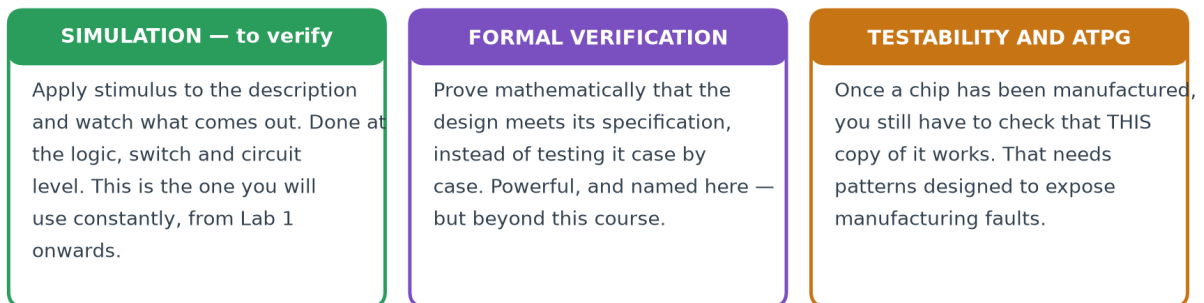
Or do not fabricate at all

The lecturer names the alternative: a field programmable gate array. An FPGA is a chip that has already been manufactured, full of configurable logic blocks that you program in your own laboratory. You get much greater flexibility, at the cost of speed.

	ASIC (fabricate a chip)	FPGA (program a device)
What you receive	masks, then wafers from a foundry	a bitstream you load into a board
Turnaround	weeks to months	minutes
Cost to start	very high — mask sets dominate	the price of the board
Cost per unit at volume	very low	stays high
Speed	highest	lower — logic is configurable, not fixed
Can you change it after?	no — respin and pay again	yes, reprogram in the lab
Used for	high volume, or when speed decides	prototypes, small runs, teaching

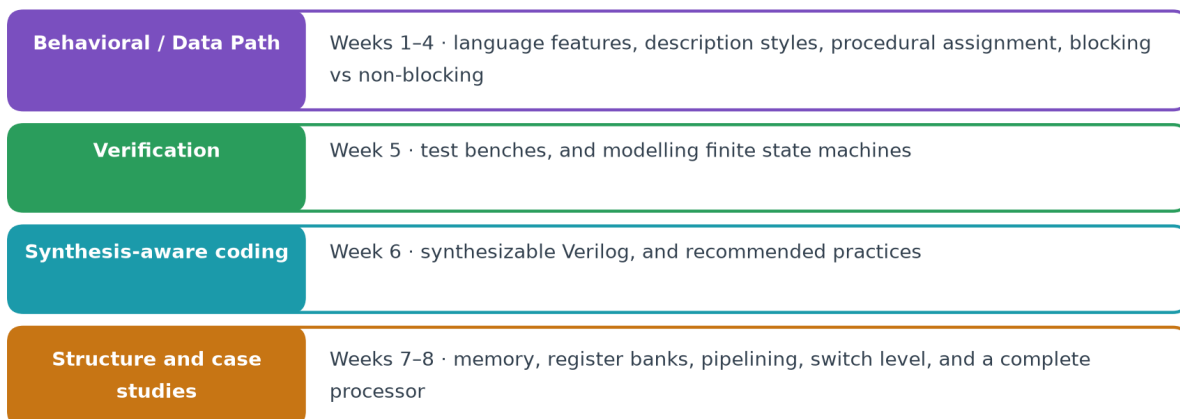
3.9 The other steps

The five rungs are not the whole flow. Three more steps run alongside them.



Simulation is the one you will use constantly, from Lab 1 onwards, and Week 5 of the course is devoted to writing test benches. Formal verification and testability analysis are named here but are beyond this course's scope.

3.10 Where the rest of the course sits



Notice what is missing: nothing after logic design. Physical design, manufacturing and test are real and essential, but in this course they are the tools' job. Your work ends at a netlist.

Part 4 · The Practical Component

Everything in this part is something you run. Every output printed below was produced by actually running the command above it, on a clean machine, with the versions named in section 4.1.

4.1 Installing the toolchain

Four programs, one for each thing the lecture says a CAD flow must do. All four are open source and run on Windows, macOS and Linux.

Four free programs, one for each thing the lecture says a CAD flow must do

ICARUS VERILOG the simulator	GTKWAVE the waveform viewer	YOSYS the synthesiser	GHDL the VHDL simulator
Compiles Verilog and runs it. This is how you check that a description behaves the way you meant. Command: iverilog, then vvp.	Draws the signals a simulation recorded, so you can look at a clock edge and see what happened. Reads the .vcd file the test bench writes.	The CAD tool of the lecture's central slide: it reads your HDL and writes a more detailed HDL. This is what turns behaviour into gates.	The same job as Icarus, for the other language — so you can compare the two on identical stimulus in Lab 5.

Tool	What it does	Used in
Icarus Verilog	compiles and simulates Verilog	Labs 1, 2, 5
GTKWave	draws the waveforms a simulation recorded	Lab 1
Yosys	synthesis — the CAD tool of the lecture	Labs 2, 3, 4
GHDL	compiles and simulates VHDL	Lab 5

Linux (Ubuntu or Debian)

```
sudo apt update
sudo apt install -y iverilog gtkwave yosys ghdl
```

macOS (Homebrew)

```
brew install icarus-verilog
brew install --cask gtkwave
brew install yosys ghdl
```

Windows

Use the Windows Subsystem for Linux. It gives you a real Ubuntu inside Windows, and every command in this workbook then works unchanged. Open PowerShell as Administrator and run:

```
wsl --install
```

Restart when it asks you to, open the Ubuntu application it installed, and then follow the Linux instructions above.

GTKWave on WSL

GTKWave is a graphical program and needs a display. Recent versions of Windows provide one automatically (WSLg) and GTKWave will simply open. If it complains that DISPLAY is not set, either update WSL with `wsl --update`, or install GTKWave natively on Windows and open the .vcd files from there — they are ordinary files in your WSL home directory.

Checking that it worked

Run all four. Each must print a version number.

the versions these labs were produced with

```
$ iverilog -V
Icarus Verilog version 12.0 (stable)

$ yosys -V
Yosys 0.33 (git sha1 2584903a060)

$ ghdl --version
GHDL 4.1.0 (Ubuntu 4.1.0+dfsg-0ubuntu2.1) [Dunoon edition]

$ gtkwave --version
GTKWave Analyzer v3.3.x
```

If a version differs

Newer is fine. A newer synthesiser may optimise slightly differently and report a different cell count from the ones printed in this workbook. That is worth noticing rather than worrying about: it demonstrates directly that optimisation has no single right answer.

Getting the lab files

```
cd ChipDesignAssociate/VerilogCourse/Lecture01_Lab
make # runs every lab in order
```

The directory contains:

Path	What is in it
rtl/counter4.v	the design, written behaviourally
rtl/tb_counter4.v	the test bench
vhdl/counter4.vhd	the same design in VHDL

vhdl/tb_counter4.vhd	the VHDL test bench
scripts/*.ys	the Yosys scripts, one per level of the flow
scripts/*.sh	the lab drivers
scripts/vcd_q.py	reads a waveform file and prints one signal
Makefile	one target per lab
out/	everything the labs produce (created on first run)

4.2 Lab 1 — simulate a counter and look at it

What this lab proves

That you can take a description of hardware, run it, and see what it does. This is the "simulation for verification" step of the design flow.

The design

rtl/counter4.v

```
// counter4 - a 4-bit up counter, written BEHAVIOURALLY.
//
// Nothing here says which gates to use, or how many. It says only
// what the output must be after each rising clock edge. Choosing
// the gates is the synthesis tool's job.
module counter4 (
    input wire      clk,
    input wire      rst,    // synchronous, active high
    output reg [3:0] q
);
    always @(posedge clk) begin
        if (rst) q <= 4'd0;
        else    q <= q + 4'd1;
    end
endmodule
```

Read it as a description, not as a program. It says: there is a four-bit register called q; on every rising edge of clk, it takes either zero or its own value plus one. It does not say how.

The test bench

rtl/tb_counter4.v (abridged — the full file prints each mismatch)

```
`timescale 1ns/1ps
module tb_counter4;
    reg      clk = 1'b0;
    reg      rst = 1'b1;
    wire [3:0] q;
    integer  errors = 0;
    integer  i;
    reg [3:0] expected;

    counter4 dut (.clk(clk), .rst(rst), .q(q));

    always #5 clk = ~clk;          // 100 MHz: a 10 ns period

    initial begin
        $dumpfile("out/counter4.vcd");
        $dumpvars(0, tb_counter4);

        @(negedge clk);
        if (q !== 4'd0) errors = errors + 1;
        rst      = 1'b0;
        expected = 4'd1;

        for (i = 0; i < 20; i = i + 1) begin
```

```

        @(negedge clk);
        if (q !== expected) errors = errors + 1;
        expected = expected + 4'd1;
    end

    if (errors == 0)
        $display("PASS  counter4  counted 0..15 and wrapped");
        $finish;
    end
endmodule

```

Why expected starts at 1, not 0

The rising edge at 5 ns happens while rst is still high, so the register is cleared there. Reset is released on the falling edge at 10 ns, which means the next rising edge — at 15 ns — is already the first counting edge.

This off-by-one is the single most common mistake in a first test bench. Getting it right means thinking about WHEN each edge happens, which is exactly the habit the rest of the course needs.

Run it

```

$ make sim
iverilog -g2012 -o out/tb_counter4.vvp rtl/counter4.v rtl/tb_counter4.v
vvp out/tb_counter4.vvp
VCD info: dumpfile out/counter4.vcd opened for output.
PASS  counter4  counted 0..15 and wrapped correctly

```

Two commands ran. The first, iverilog, compiled the design and the test bench into a simulation program. The second, vvp, ran it.

Look at the waveform

```
$ make wave
```

GTKWave opens. Drag clk, rst and q from the signal list into the wave window. You should see q change immediately after each rising clock edge, counting 1, 2, 3 ... 15, 0, 1 ...

What to notice in the waveform

q is xxxx — unknown — until the first clock edge. A Verilog reg holds x until something drives it, and a real flip-flop likewise powers up in an unpredictable state. That is why every design has a reset.

q changes AFTER the clock edge, not at it. That delay is the flip-flop's clock-to-Q time, and it is the first thing that eats into your clock period.

4.3 Lab 2 — watch a CAD tool rewrite your Verilog

What this lab proves

The central claim of Lecture 01: that a CAD tool transforms an HDL input into an HDL output containing more detail about the hardware, without changing what the circuit does.

Step 1 — the transformation, twice

```
$ make transform
```

This prints three descriptions of the same counter. The first is what you wrote:

1 — behavioural, written by a human

```
always @(posedge clk) begin
    if (rst) q <= 4'd0;
    else     q <= q + 4'd1;
end
```

The second is what Yosys produced after synthesis. It is still Verilog, but the increment has become a named component with its own wire:

2 — register transfer level, written by Yosys

```
wire [3:0] _0_;
assign _0_ = q + 4'h1;           // an incrementer
always @(posedge clk)           // a 4-bit register
    if (rst) q <= 4'h0;          // with a synchronous reset
    else   q <= _0_;
```

The third is what Yosys produced after mapping to gates. Still Verilog, and now every operation is a single logic function:

3 — gate level, written by Yosys

```
assign _02_[0] = ~q[0];
assign _00_ = q[0] & q[1];
assign _01_ = ~(q[2] & _00_);
assign _03_[3] = ~(q[3] ^ _01_);
assign _03_[1] = q[0] ^ q[1];
assign _03_[2] = q[2] ^ _00_;
```

Look at what just happened

All three files are Verilog. The tool did not compile your design into something else — it re-described it, twice, each time in more detail. The third is a netlist: gates, and the wires between them.

Step 2 — and it still behaves the same

A more detailed description is only useful if it is still the same circuit. Run the SAME test bench against the gate netlist:

```
$ make prove

--- 1. the behavioural description (what the designer wrote) ---
PASS  counter4 counted 0..15 and wrapped correctly

--- 2. synthesise it down to gates ---
     wrote out/counter4_gates.v (33 lines)

--- 3. the SAME test bench, run against the gate netlist ---
PASS  counter4 counted 0..15 and wrapped correctly

--- 4. compare the two waveforms, cycle by cycle ---
IDENTICAL - 22 transitions, same value at the same time.
The synthesiser changed the DESCRIPTION, not the BEHAVIOUR.
```

One test bench, two very different descriptions, identical output at every instant. Nothing in the lecture has to be taken on trust after this.

How the gate netlist can be simulated at all

Yosys emits primitives with names like `$_AND_` and `$_SDFF_PP0_`. Yosys ships a file, `simcells.v`, that defines them as ordinary Verilog modules; the lab script finds it and passes it to Icarus along with the netlist.

4.4 Lab 3 — count the cells at each level

What this lab proves

That detail really does increase down the ladder, in a way you can put a number on.

```
$ make ladder
```

LEVEL	CELLS	WHAT THEY ARE
1 Behavioural (source)	-	10 lines of Verilog - no cells
2 Data path (RTL)	2	\$add x1 \$sdff x1
3 Logic (gates)	10	\$_AND_ x1 \$_NAND_ x1 \$_NOT_ x1 \$_SDFF_PP0_ x4 \$_XNOR_ x1 \$_XOR_ x2
same, for an FPGA	8	\$_SDFF_PP0_ x4 \$lut x4

Read it downwards. Ten lines of behaviour became two RTL components — one adder and one four-bit register, which is exactly what section 3.6 said "data path design" means. Those two became ten gate-level cells: four flip-flops, one per bit, and six logic gates to compute the next value.

Yosys 0.33, on rtl/counter4.v — reproduce it with `make ladder`

Level of the flow	Cells	What they are
1 · Behavioural source	—	10 lines of Verilog — no cells yet
2 · Data path (RTL)	2	1 adder, 1 four-bit register
3 · Logic, for an ASIC	10	4 flip-flops + AND, NAND, NOT, XNOR, XOR ×2
3 · Logic, for an FPGA	8	4 flip-flops + 4 look-up tables

Same circuit every time. Only the vocabulary changes — and the number of words needed to say it.

the same table, as the slide shows it

A detail worth noticing

One \$sdff at RTL became four \$_SDFF_PP0_ at gate level. At RTL a register is one component that happens to be four bits wide; at gate level each bit is its own flip-flop. The vocabulary changed, not the circuit.

4.5 Lab 4 — ASIC mapping against FPGA mapping

What this lab proves

That the same Verilog produces different hardware depending on what you are building it for — the flexibility-for-speed trade the lecturer names when he introduces FPGAs.

```
$ make target
```

```

=== mapped to logic GATES (the ASIC route) ===
Number of cells:      10
$_AND_                1
$_NAND_               1
$_NOT_                1
$_SDFF_PP0_          4
$_XNOR_               1
$_XOR_                2

=== mapped to 4-input LOOK-UP TABLES (the FPGA route) ===
Number of cells:      8
$_SDFF_PP0_          4
$lut                  4

```

The flip-flops are the same either way — one per bit of the counter. What changes is the combinational logic: six dedicated gates for silicon you will manufacture, or four look-up tables inside a chip you can buy today.

	ASIC	FPGA
What you receive	masks, then wafers from a foundry	a bitstream you load into a board
Turnaround	weeks to months	minutes
Cost to start	very high — mask sets dominate	the price of the board
Cost per unit at volume	very low	stays high
Speed	highest	lower — logic is configurable, not fixed
Changeable after?	no — respin and pay again	yes, reprogram in the lab

Why a LUT replaces several gates

A four-input look-up table is a small memory holding sixteen bits. Program those sixteen bits and it can compute ANY function of four inputs. So a chain of two or three gates with four inputs between them collapses into one LUT — which is why the FPGA cell count is lower even though the chip is slower.

4.6 Lab 5 — the same design in two languages

What this lab proves

That the choice between Verilog and VHDL is a matter of style, not of what the hardware can do.

```
$ make langs

--- Verilog (Icarus) ---
PASS counter4 counted 0..15 and wrapped correctly

--- VHDL (GHDL) ---
PASS counter4 counted 0..15 and wrapped correctly

--- the whole run, including time zero ---
they differ, and ONLY here:
1,2c1
<      0.000 ns   xxxx
<      5.000 ns   0000
---
>      0.000 ns   0000

--- from the end of reset onwards (15 ns) ---
IDENTICAL - 20 transitions, same value at the same time
```

The one difference is the most useful part of the lab

A Verilog reg holds x — unknown — until something drives it, so q reads xxxx until the reset edge at 5 ns clears it. The VHDL signal was declared with an initial value, so it reads 0000 from time zero.

The described hardware is identical. A real flip-flop also powers up in an unpredictable state, which is exactly why every design has a reset — and why the Verilog behaviour is arguably the more honest model of the two.

How the comparison is done

Icarus and GHDL write their waveform files with different timescales and different internal identifiers, so the raw files cannot be compared directly. The lab includes a small reader that normalises both to nanoseconds and to plain binary:

```
$ python3 scripts/vcd_q.py out/counter4.vcd q | head -4
0.000 ns   xxxx
5.000 ns   0000
15.000 ns  0001
25.000 ns  0010
```

Two details in that script are worth knowing about, because both are easy to get wrong. Times are kept as whole femtoseconds rather than floating-point nanoseconds, because comparing floating-point times across two different timescales is unreliable. And a vector is padded on the left with 0, except that a

value starting with x or z extends with that character — which is why the first line above reads xxxx and not 000x.

A note on GHDL back ends

GHDL has several code generators. With the mcode back end — the usual one on Ubuntu — `ghdl -e` does not write an executable file and you run the simulation with `ghdl -r`. With the LLVM or GCC back ends, `ghdl -e` produces a binary you run directly. The lab script uses `ghdl -r`, which works with all of them.

Part 5 · Exercises

There are 44 exercises below, in four groups. Worked solutions to every one of them begin on the next page — try each question before you look.

The ones that matter most

If you are short of time, do the concept questions on the netlist (3, 4, 8 and 9) and the whole of the lab group. Those are the ideas the rest of Module 2 is built on.

Concept questions

1. Name the five levels of the design flow, in order, from a design idea down to a finished chip.
2. What are the input and the output of a CAD tool, and how do they differ?
3. Define a netlist.
4. Why is a netlist represented as a graph rather than as a drawing?
5. What does "register transfer" refer to?
6. Give four ways of specifying the behaviour of a circuit.
7. What do all four of those have in common?
8. Why is a netlist also called a structural description?
9. State synthesis in one sentence, using the words behavioural and structural.
10. What is a standard cell?
11. Why does a standard cell library exist at all?
12. Name three logic-optimisation goals that conflict with one another.
13. State Moore's Law. Is it a law of physics?
14. Why are the shapes in a layout all regular polygons, mostly rectangles?
15. Give one advantage of an FPGA over an ASIC, and one the other way round.
16. Name three steps of the design flow that are not rungs of the ladder.
17. Why does the flow contain a verification step at every level rather than only one at the end?
18. What is the difference between combinational and sequential logic?

The language

19. Why is Verilog not a programming language, even though it looks like one?

- 20. What happens if you swap two assign statements in a Verilog module, and why?
- 21. Name two HDLs other than Verilog and VHDL, and say what each is for.
- 22. Will a synthesis tool produce different gates from a VHDL description than from an equivalent Verilog one?
- 23. What is a test bench, and why may it use constructs that are not synthesisable?
- 24. A test bench has no ports. Why not?

From the labs

- 25. Run `make sim` and then `make wave`. In the waveform, what is the value of `q` before the first clock edge, and why?
- 26. In the waveform, does `q` change at the clock edge or just after it?
- 27. Run `make transform`. How many of the three printed descriptions are Verilog?
- 28. Run `make ladder`. How many cells does the counter have at RTL, and what are they?
- 29. One `$sdff` at RTL became four `$_SDFF_PP0_` at gate level. Explain.
- 30. Change the counter to 8 bits. Re-run `make ladder`. Did the gate count double?
- 31. Make the reset asynchronous — `always @(posedge clk or posedge rst)` — and compare the gate netlists. What changed?
- 32. Write a down-counter that loads 15 on reset, and compare it with the up counter.
- 33. Try `abc -lut 6` instead of `abc -lut 4`. Why do larger look-up tables not help here?
- 34. Map the 8-bit counter for an FPGA. Is it bigger or smaller than the ASIC mapping?
- 35. Open `out/counter4_behav.vcd` and `out/counter4_gates.vcd` in GTKWave together and align them. What do you expect to see?
- 36. Run `make langs`. Where do the Verilog and VHDL runs differ, and why?

Applying it

- 37. A colleague says "synthesis compiles Verilog into gates, like gcc compiles C into machine code". What is wrong with the analogy?

- 38.** Your design meets its area budget but misses its clock target. Name two things you could change, and what each would cost.
- 39.** You are asked to build 200 units of a product with a tight deadline and a modest budget. ASIC or FPGA? Justify it.
- 40.** Why can two synthesis runs of the same Verilog produce different cell counts?
- 41.** You read in a datasheet that a chip is built on a "3 nm" process. What does the number mean?
- 42.** If Moore's Law is slowing, why does the design flow in this lecture still apply?
- 43.** Where in the flow does the course you are taking stop, and why?
- 44.** Someone shows you a Verilog file full of named gate instances and asks whether it is behavioural or structural. How do you answer?

Part 5 · Solutions

Each solution is numbered to match the exercise. Where a solution quotes a number, that number was measured by running the command, not estimated.

Concept questions

1. *Name the five levels of the design flow, in order, from a design idea down to a finished chip.*

Behavioral design, data path design (register transfer level), logic design, physical design, manufacturing. The lecturer draws them as a ladder with "design idea" above the first and "chip / board" below the last.

2. *What are the input and the output of a CAD tool, and how do they differ?*

Both are hardware descriptions written in an HDL. The output contains more detailed information about the hardware than the input. Nothing is compiled into a program; the design is simply re-described in more concrete terms.

3. *Define a netlist.*

A directed graph whose vertices are components and whose edges are the interconnections between them. The whole graph is the design.

4. *Why is a netlist represented as a graph rather than as a drawing?*

Because a graph is a data structure a program can hold in memory and transform. Every CAD step after synthesis — optimisation, technology mapping, placement, routing — is an operation on that graph.

5. *What does "register transfer" refer to?*

That on each clock edge, values are transferred between registers, possibly passing through some combinational logic on the way. The name describes what the level models.

6. *Give four ways of specifying the behaviour of a circuit.*

A Boolean expression; a truth table; a finite-state machine, as a state transition diagram or table; and a high-level algorithm written much as you would write C.

7. *What do all four of those have in common?*

None of them says how the circuit is BUILT — only what it must DO. That is what makes them behavioural, and it is why a synthesis tool is free to build them in whichever way is cheapest under your constraints.

8. *Why is a netlist also called a structural description?*

Because it names the parts and says how they are wired together, rather than describing the result. The lecturer states the equivalence explicitly: a netlist specification is also referred to as structural design.

9. *State synthesis in one sentence, using the words behavioural and structural.*
You write behavioural Verilog; the tool emits structural Verilog.

10. *What is a standard cell?*

A pre-designed circuit module — a gate, a flip-flop, a small multiplexer — held in a library as a symbol to place, a truth table to simulate, a timing model, and a finished piece of layout.

11. *Why does a standard cell library exist at all?*

Laying out even a NAND gate correctly takes a skilled engineer a long time. Doing it once, perfectly, and reusing it a million times is the only way a billion-transistor design is economically possible.

12. *Name three logic-optimisation goals that conflict with one another.*

Minimise the number of gates (area); minimise the number of gate levels (delay); minimise signal transition activity (dynamic power). Improving one usually makes at least one of the others worse.

13. *State Moore's Law. Is it a law of physics?*

That the number of transistors on a chip doubles roughly every two years. It is not a law of physics: it was an observation about what the industry had been doing, and it kept holding largely because it became a planning target for the whole supply chain.

14. *Why are the shapes in a layout all regular polygons, mostly rectangles?*

Because they are printed photographically, layer by layer, onto a silicon wafer. Regular shapes expose cleanly and etch predictably; awkward ones do not.

15. *Give one advantage of an FPGA over an ASIC, and one the other way round.*

An FPGA can be programmed in your own laboratory in minutes and reprogrammed afterwards, with no mask costs. An ASIC is faster, smaller and far cheaper per unit at volume, because its logic is built from dedicated transistors rather than configurable ones.

16. *Name three steps of the design flow that are not rungs of the ladder.*

Simulation for verification; formal verification; testability analysis and test pattern generation.

17. *Why does the flow contain a verification step at every level rather than only one at the end?*

Because each level is produced from the one above it by a different tool, and a mistake is far cheaper to find at the step that made it than three steps later. It is also why fabrication, which is statistical rather than exact, needs test patterns of its own.

18. *What is the difference between combinational and sequential logic?*

Combinational logic has no memory: its outputs depend only on its present inputs. Sequential logic has memory, held in registers and updated on a clock edge, so its outputs depend also on what happened before.

The language

19. *Why is Verilog not a programming language, even though it looks like one?*

Because its statements are not events in time. They are a description of hardware that exists all at once and stays there. Three assign statements describe three wires, all permanently driven; reordering them changes nothing.

20. *What happens if you swap two assign statements in a Verilog module, and why?*

Nothing. Each assign describes a piece of wire that exists permanently, so there is no "first" and no "next" — only what is connected to what.

21. *Name two HDLs other than Verilog and VHDL, and say what each is for.*

SystemVerilog, a large superset of Verilog adding classes, constrained random stimulus and assertions, aimed mostly at verification. SystemC, a C++ class library used to model whole systems before the hardware is written.

22. *Will a synthesis tool produce different gates from a VHDL description than from an equivalent Verilog one?*

No. Both describe the same hardware, and the tool produces the same gates from either. The choice is a matter of house style, existing code and local convention.

23. *What is a test bench, and why may it use constructs that are not synthesisable?*

A piece of HDL whose job is to drive a design with stimulus and check what comes out. It is never turned into hardware, so delays, loops and \$display are all legitimate in it.

24. *A test bench has no ports. Why not?*

Because nothing drives it and nothing reads it — it sits at the top of the hierarchy and instantiates the design under test inside itself.

From the labs

25. *Run `make sim` and then `make wave`. In the waveform, what is the value of `q` before the first clock edge, and why?*

xxxx — unknown. A Verilog reg holds x until something drives it, and the first rising edge (at 5 ns, with rst still high) is what clears it. A real flip-flop likewise powers up in an unpredictable state, which is why every design has a reset.

26. *In the waveform, does q change at the clock edge or just after it?*

Just after. The delay is the flip-flop's clock-to-Q time, and it is the first thing that eats into the available clock period.

27. *Run `make transform` . How many of the three printed descriptions are Verilog?*

All three. That is the point of the exercise: the tool re-described the design twice, each time in more detail, and never left the language.

28. *Run `make ladder` . How many cells does the counter have at RTL, and what are they?*

Two: one `$add` and one `$sdff`. That is literally "an adder and a register", which is what data path design means.

29. *One `$sdff` at RTL became four `$_SDFF_PP0_` at gate level. Explain.*

At RTL a register is a single component that happens to be four bits wide. At gate level each bit is its own flip-flop. The vocabulary changed, not the circuit.

30. *Change the counter to 8 bits. Re-run `make ladder` . Did the gate count double?*

No — it more than doubled. The 4-bit version is 10 cells (4 flip-flops and 6 gates); the 8-bit version is 22 cells (8 flip-flops and 14 gates). The flip-flops scale exactly with width, but the carry chain that computes "plus one" grows faster, because each higher bit must test more of the lower bits.

31. *Make the reset asynchronous — always `@(posedge clk or posedge rst)` — and compare the gate netlists. What changed?*

The total is still 10 cells, and the combinational logic is unchanged. What changed is the flip-flop: `$_SDFF_PP0_` (synchronous reset) became `$_DFF_PP0_` (asynchronous reset). The reset moved out of the logic and into the flip-flop itself.

32. *Write a down-counter that loads 15 on reset, and compare it with the up counter.*

Also 10 cells, but a different mix: 4 flip-flops, 1 NOT, 2 OR and 3 XNOR, against the up counter's 1 AND, 1 NAND, 1 NOT, 1 XNOR and 2 XOR. The flip-flop is `$_SDFF_PP1_` — one that resets to 1 rather than 0 — because the reset value is now `4'd15`. Counting down costs the same as counting up.

33. *Try `abc -lut 6` instead of `abc -lut 4` . Why do larger look-up tables not help here?*

The count stays at 8 cells — 4 flip-flops and 4 LUTs. Each next-state bit of a 4-bit counter already depends on at most four inputs, so a 4-input LUT is big enough. Making the LUTs larger cannot reduce a count that is already one LUT per bit.

34. *Map the 8-bit counter for an FPGA. Is it bigger or smaller than the ASIC mapping?*

Smaller in cell count: 18 cells (8 flip-flops and 10 LUTs) against 22 for the gate mapping. Cell count is not area, though — a LUT is a small memory and occupies far more silicon than the gate it imitates, which is exactly the flexibility-for-speed trade.

35. *Open out/counter4_behav.vcd and out/counter4_gates.vcd in GTKWave together and align them. What do you expect to see?*

Identical traces. make prove already compared them mechanically and reported 22 transitions at the same values and the same times; looking at them is the human confirmation of the same fact.

36. *Run make langs . Where do the Verilog and VHDL runs differ, and why?*

Only at time zero. The Verilog reg reads xxxx until the reset edge clears it; the VHDL signal was declared with an initial value and so reads 0000 from the start. From the end of reset onwards the two are identical for all 20 transitions.

Applying it

37. *A colleague says "synthesis compiles Verilog into gates, like gcc compiles C into machine code". What is wrong with the analogy?*

gcc produces something in a different kind of language — instructions for a processor to execute in sequence. Synthesis produces another hardware description, in the same language, of a thing that exists all at once. Nothing is ever executed.

38. *Your design meets its area budget but misses its clock target. Name two things you could change, and what each would cost.*

Add pipeline registers, which shortens the longest logic path but costs area and adds latency. Or let the tool duplicate logic so parts run in parallel, which costs area and switching power. Both trade area or power for speed, as the triangle predicts.

39. *You are asked to build 200 units of a product with a tight deadline and a modest budget. ASIC or FPGA? Justify it.*

FPGA. Mask costs dominate at low volume and would never be recovered over 200 units, and the turnaround is minutes rather than months. You accept lower clock speed and higher unit cost in exchange.

40. *Why can two synthesis runs of the same Verilog produce different cell counts?*

Because there is no single best answer. The tool searches for a solution satisfying the constraints it was given, and different constraints, different versions or different optimisation effort will land on different points of the area/speed/power trade.

41. *You read in a datasheet that a chip is built on a "3 nm" process. What does the number mean?*

Essentially nothing dimensional. Node names are now marketing labels: no feature on a "3 nm" chip measures 3 nm. They indicate a generation, not a measurement — useful for comparing roadmap steps, not for computing anything.

42. *If Moore's Law is slowing, why does the design flow in this lecture still apply?*

Because the flow is about managing complexity, not about transistor size. Chiplets, 3-D stacking and specialised accelerators all still need a behavioural description turned into a netlist and then into layout, with verification at each level.

43. *Where in the flow does the course you are taking stop, and why?*

At logic design. Physical design, manufacturing and test are real and essential but are somebody else's job — and in practice the tools'. A Verilog designer's deliverable is a netlist.

44. *Someone shows you a Verilog file full of named gate instances and asks whether it is behavioural or structural. How do you answer?*

Structural. It names the parts and wires them together rather than describing the result, which is exactly what a netlist is — and a netlist written as text is a structural description.

Part 6 · Reference

6.1 The flow at a glance

Level	A component is...	Produced by	Checked by
Behavioral	a statement	a human	simulation
Data path (RTL)	register, adder, mux	synthesis	simulation, equivalence
Logic	gate or flip-flop	technology mapping	simulation, static timing
Physical	a placed cell	place and route	timing, DRC, LVS
Manufacturing	a polygon	the foundry	wafer test

6.2 Commands used in this workbook

Command	What it does
iverilog -g2012 -o out/x.vvp a.v b.v	compile Verilog into a simulation
vvp out/x.vvp	run that simulation
gtkwave out/x.vcd	view the waveform it recorded
yosys -s script.js	run a synthesis script
ghdl -a --workdir=out/ghdl x.vhd	analyse VHDL
ghdl -r --workdir=out/ghdl tb --vcd=out/x.vcd	elaborate and run it
make sim	Lab 1
make transform / make prove	Lab 2
make ladder	Lab 3
make target	Lab 4
make langs	Lab 5
make clean	delete everything under out/

6.3 Yosys commands used in the scripts

Command	What it does
read_verilog f.v	parse a Verilog file
hierarchy -top m	pick the top module, resolve the hierarchy
proc	turn always blocks into registers and logic
opt	general optimisation
techmap	map internal operators onto simple cells
abc -g AND,OR,XOR,...	map logic onto a given set of gates
abc -lut 4	map logic onto 4-input look-up tables
opt_clean	remove unused cells and wires
write_verilog -noattr f.v	write the netlist back out as Verilog
stat	print how many cells the design now has

Reading a Yosys statistics block

Yosys prints statistics every time the stat command runs, and a script may run it more than

once. Only the LAST block describes the finished netlist — which is why scripts/stat.sh extracts just that one.

6.4 Glossary

ASIC — Application-Specific Integrated Circuit — a chip manufactured for one design.

Behavioural description — A description of what a circuit does, saying nothing about which parts build it.

CAD tool — A program that reads a hardware description and writes a more detailed one.

Cell — One component of a netlist: at gate level, a gate or a flip-flop.

Clock-to-Q — The delay between a clock edge arriving at a flip-flop and its output changing.

Combinational logic — Logic whose outputs depend only on its present inputs.

Design flow — The standardized sequence of steps from a specification to a manufactured chip.

FPGA — Field Programmable Gate Array — a manufactured chip full of configurable blocks you program yourself.

HDL — Hardware Description Language. Verilog and VHDL are the two in common use.

LUT — Look-Up Table — a small memory inside an FPGA that can compute any function of its inputs.

Netlist — A directed graph whose vertices are components and whose edges are interconnections.

RTL — Register Transfer Level — the level at which the components are registers, adders, multiplexers and buses.

Sequential logic — Logic with memory, updated on a clock edge.

Standard cell — A pre-designed module held in a library as symbol, behaviour, timing model and layout.

Structural description — A description naming the parts and how they are wired. A netlist, written as text.

Synthesis — The step that turns a behavioural description into a structural one.

Technology mapping — Rewriting a netlist in terms of the cells a particular library or FPGA actually provides.

Test bench — HDL that drives a design with stimulus and checks the result. Never synthesised.

VCD — Value Change Dump — the file format a simulator writes so a waveform viewer can draw the result.

6.5 Where to go next

Lecture 02, Design Representation, takes the idea this lecture left implicit — that the same design can be looked at from different angles — and makes it the subject. It arranges the behavioural, structural and physical views as the three arms of a Y-diagram, with the highest level of abstraction at the outside and the finished layout at the centre. Every design step is then a move along one arm, or a jump from one arm to another.

Keep the lab directory

Everything from Lecture 02 onwards builds on the same toolchain. The counter, the test bench and the Makefile in Lecture01_Lab are a working template you can copy for any later design.